PRD: Usage-Based Billing / Metering Service

Working name: MeterLedger · Full-stack service that ingests usage events, meters them and produces tiered invoices · Completed PRD for a 4-week first-year OJT project

1. Header

Field	Value
Project	Usage-Based Billing / Metering Service (working name **MeterLedger**)
ID/Track	PST_AD_162 · Application Development (full stack)
Batch / Level	2026 Level 4 (Core)
Duration	4 weeks (each week split between learning and building)
Student/Team	2-student team (names in Sign-off). Suggested split: Student A owns ingestion, metering and the database; Student B owns pricing/invoices UI, the tenant portal and the analysis script. Both write the pricing function and its tests together in Week 1.
PRD version	v1.0 (completed from catalog draft v0.1)
Status	Ready for kickoff
Platform	Web app with a real backend: Python FastAPI server + SQLite + Bootstrap/vanilla JS frontend, plus a pandas/matplotlib analysis script. Payments are simulated; Stripe test mode is optional stretch.

2. Overview

MeterLedger is the billing back office for a small company that sells an API (for example an SMS or email API). Each customer company is a **tenant**. Every time a tenant uses the product, the tenant's server sends a small JSON **usage event** to MeterLedger, such as "ShopKart made 500 API calls at 10:21". MeterLedger stores each event exactly once, adds them up per tenant, per meter, per month, applies the tenant's **pricing plan** (a free tier, then paid tiers) and produces an **invoice** with one line per tier.

The admin web console shows tenants, their usage this month, a live "estimate so far" for each tenant, and the invoice list where drafts are generated, finalized and marked paid (a mock payment). Tenants log in with their API key to a small portal that shows their own usage and invoices. An event simulator, including a "retry storm" button, proves that sending the same event many times never charges twice. A Python script turns the exported usage into charts, including the catalog's box plot of usage per tenant.

- **Industry:** FinTech
- **Theme:** Usage Metering & Billing Infrastructure
- **One-line pitch:** "For API startups who need to bill customers for what they actually use, MeterLedger turns raw usage events into correct tiered invoices, unlike month-end spreadsheets that double-count retries and miss tier boundaries."

> **Scoped for 4 weeks.** Production billing systems stream events through Kafka or Kinesis, handle taxes, currencies, proration, refunds and real card payments. Here events arrive over plain HTTP into SQLite, there is one currency (INR) and no tax, and "payment" is a button. Students still build the parts that make billing *correct*: idempotent ingestion, aggregation by month, tiered pricing with exact integer money, and an invoice lifecycle. Stripe is an optional stretch in test mode only.

3. Problem statement

Small API companies usually bill from server logs: someone exports a CSV at month end, sums it in a spreadsheet and types the totals into an invoice. Two things go wrong. Client libraries **retry** when the network

is slow, so the same usage is often logged two or three times. And tiered prices ("first 10,000 calls free, then ₹20 per 1,000") are easy to apply wrongly at the boundaries.

Concrete pain today: On the 1st of the month, Kavya, who runs finance at a four-person SMS API startup, spends a full day building 12 invoices in Excel. ShopKart then disputes its bill: it was charged for 182,000 API calls, but its own logs say 135,400. The difference was network retries counted twice. Kavya has no way to prove which number is right, so she issues a ₹900 credit and loses another afternoon.

What MeterLedger fixes: every event carries a unique `event_id`, so a retry is recognised and ignored; totals are computed by the database, not by hand; and the same tested pricing function produces every invoice line.

How we'll know it's solved: (1) a retry-storm test sending about 100,000 requests (20,000 unique events, each repeated 1 to 9 times) ends with usage totals exactly equal to the sum of the 20,000 unique events; (2) all six hand-computed invoice cases in Section 15 match to the paisa; (3) generating all invoices for a month takes one click instead of a day.

4. Users and roles

	Kavya, Billing Admin	Arjun, Tenant Developer	Sameer, Tenant Finance
Context	Runs finance and plans at the API company	Integrates the tenant's server with the metering API	Pays the bill at the tenant company
Goal	Correct invoices for all tenants in minutes	Send events reliably, even with retries	Understand what they are being charged for
Frustration	Spreadsheets, disputes, tier mistakes	Fears retries will double-charge	Invoices show one total with no breakdown
Success	One-click draft invoices that match hand checks	Duplicate events come back as "duplicate", not charged	Invoice shows each tier, units and rate

Role capabilities matrix

Capability	Admin	Tenant (own API key)	Not logged in
Send usage events for a tenant	✔ (any tenant, for testing)	✔ (own tenant only)	✘
View usage and invoices	✔ all tenants	✔ own only	✘
Create tenants, change a tenant's plan	✔	✘	✘
Create / edit pricing plans	✔ (only plans not yet used on an invoice)	✘ (can view the price list)	✔ view price list only
Generate, finalize, void, mark paid invoices	✔	✘	✘
Export usage CSV, run retry storm	✔	✘	✘

5. Scope

Must have (MVP, graded)

1. **Record usage events**: `POST /api/events` and a batch endpoint (up to 1,000 events) with validation and **idempotency** on `(tenant_id, event_id)`.
2. **Meters**: two seeded meters, `api_calls` and `emails_sent`, aggregated by **sum**.
3. **Monthly sum**: usage per tenant, per meter, per calendar month (UTC), plus a daily breakdown.
4. **Pricing plans with tiers**: base fee plus graduated tiers per meter (a free tier, then paid tiers). A flat "units × price" plan is simply a plan with one tier.
5. **Invoice table**: generate draft invoices for a month, one line per tier; finalize (locks it and assigns a number); mark paid (mock); void.

6. **Admin console** (tenants, tenant detail with usage chart, plans, invoices, invoice detail) and a **tenant portal** (own usage and invoices).
7. **Sample data**: seed script with 3 plans, 8 tenants and 3 months of events generated from the AWS/GCP simulated metering CSV; `sample-data.json`.
8. **Analysis script** (small, required because Section 15 expects it): monthly totals per tenant and a box plot of tenant usage with pandas and matplotlib.

Good to have

- "Estimate so far" invoice preview for the current month on the tenant detail and portal.
- Event simulator page with a **retry storm** button and a running accepted/duplicate counter.
- Price calculator on the Plans page ("what would 135,400 calls cost on Starter?").
- CSV export of invoices; printable invoice page.
- Usage alert banner when a tenant passes 80% of its free tier.

Stretch (optional self-study)

- Stripe **test mode**: push a finalized invoice to Stripe as invoice items and open the hosted invoice page. Test keys only, never live keys.
- Volume pricing (all units at the rate of the tier reached) as a second pricing model, compared with graduated pricing.
- Late events after finalization recorded as adjustment lines on the next invoice.
- Deeper EDA on the metering CSV: usage seasonality, heavy-tail tenants, revenue concentration.

Out of scope

1. Kafka, Kinesis or any message queue; events arrive over HTTP only.
2. Real payments, card storage, taxes (GST), multiple currencies, refunds, proration for mid-month plan changes.
3. Tenant self sign-up, password accounts, OAuth or emailing invoices.
4. Gauge-type meters (for example storage GB-hours) and "max" or "unique count" aggregation.

6. Functional requirements

ID	Requirement	How it is tested
FR-01	Accept a usage event `{event_id, meter, quantity, timestamp}` for the tenant identified by the API key; store it once.	201 `accepted`; row appears in `usage_events`.
FR-02	An event with the same `event_id` and identical data is ignored and reported as duplicate; same id with different data is rejected.	Second send → 200 `duplicate`; changed quantity → 409.
FR-03	Batch ingest of up to 1,000 events returns counts of accepted, duplicates and per-item rejections; good events are stored even if some fail.	Batch with 3 good, 1 duplicate, 1 bad meter → 3/1/1.
FR-04	Validate events: known meter, integer quantity 1 to 1,000,000, timestamp not more than 5 minutes in the future, month not already finalized.	Each invalid case returns the documented 400/409 code.
FR-05	Return monthly totals per meter and daily totals for a tenant and month.	Sum equals SQL check query on seed data.
FR-06	Price usage with the plan's graduated tiers using integer paise and the rounding rule in Section 10.	Six worked cases in Section 15 match exactly.

ID	Requirement	How it is tested
FR-07	Admin generates draft invoices for a month (all tenants or one); re-generating replaces draft lines only.	Generate twice → still one invoice per tenant per month.
FR-08	Admin finalizes an invoice: it becomes read-only and gets a number `INV-YYYYMM-NNNN`; mark paid and void follow Section 10.	Edit or regenerate after finalize → 409.
FR-09	Admin manages tenants (create with one-time API key display, change plan) and plans (create, view tiers).	New tenant key works once copied; plan change shows on next draft.
FR-10	Tenant portal shows only that tenant's usage and invoices.	Tenant key requesting another tenant's invoice → 404.
FR-11	Export usage as CSV (tenant, meter, day, quantity) for the analysis script.	Opens in pandas; totals match the API.
FR-12	Retry-storm script proves zero double counting.	Section 15 metric passes.

User stories

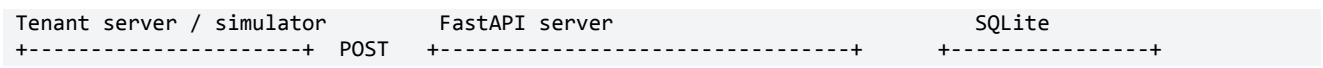
ID	As a...	I want to...	So that...	Priority
US-01	Tenant developer	send usage events with my API key	my company's usage is recorded	Must
US-02	Tenant developer	safely retry an event after a timeout	my company is never charged twice	Must
US-03	Billing admin	see each tenant's usage this month	I know who is using what	Must
US-04	Billing admin	define a plan with a free tier and paid tiers	pricing matches our price page	Must
US-05	Billing admin	generate and finalize all invoices for a month	billing takes minutes, not a day	Must
US-06	Tenant finance	see each tier, units and rate on my invoice	I can check the bill myself	Must
US-07	Billing admin	mark an invoice as paid	I can track who still owes money	Must
US-08	Tenant finance	see an estimate for this month so far	there are no surprises	Good
US-09	Billing admin	run a retry storm from the UI	I can show customers billing is safe	Good
US-10	Billing admin	send an invoice to Stripe test mode	I learn how real payment flows work	Stretch

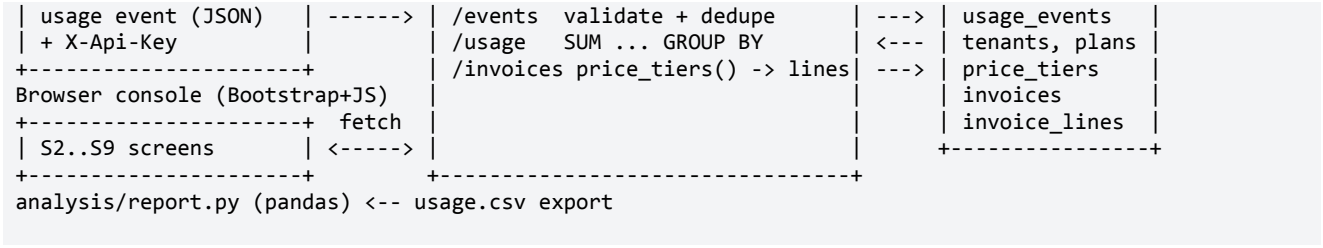
Key screens (UI requirements)

#	Screen	Key elements	Who
S1	Sign in	Paste API key or admin key; role detected from `GET /api/me`; link to public price list	All

#	Screen	Key elements	Who
S2	Overview	KPI cards: events today, active tenants, estimated revenue this month, open invoices; recent events table (last 20)	Admin
S3	Tenants	Table: name, plan, API calls this month, emails this month, estimate; search; New tenant (key shown once in a modal with Copy)	Admin
S4	Tenant detail	Plan and Change plan; month picker; totals per meter; daily usage bar chart (Chart.js); free-tier progress bar; estimate-so-far invoice preview	Admin
S5	Plans & pricing	Plan cards with tier tables; create-plan form with add-tier rows; price calculator	Admin; read for all
S6	Invoices	Month picker; Generate drafts; table (number, tenant, total, status pill); status filter; Finalize / Mark paid / Void actions; totals row; Export CSV	Admin
S7	Invoice detail	Printable layout: tenant, period, plan, line items (meter, tier, units, rate, amount), base fee, total, status, timeline (drafted, finalized, paid)	Admin; tenant (own)
S8	Event simulator	Tenant select; single-event form with "Generate event_id"; batch generator; Retry storm (N unique × up to R repeats); live counters accepted / duplicate / rejected; final check "usage total = expected"	Admin
S9	Tenant portal	My usage this month (meters, free-tier bar, daily chart), my invoices list and detail, my API key (masked) and sample `curl`	Tenant

UI states: every screen has loading (spinner or skeleton), empty ("No events yet this month. Send one from the simulator or with the sample curl."), error (toast with the API's `detail`) and success states. Money is always shown as `₹1,800.00` with two decimals. Status is shown as text plus colour (Draft grey, Finalized blue, Paid green, Void red). Tables become cards under 576 px. Lighthouse accessibility ≥ 90 on S3, S6 and S7.





7. Tech stack

Layer	Choice	Why (for a first-year student)
Backend	Python 3.11+, **FastAPI**, Uvicorn	Builds on Sem 1 Python. Pydantic validation turns most of Section 12 into model rules; the `/docs` page lets students test the API before the UI exists.
Database	**SQLite** via Python's built-in `sqlite3`	One file, no install. Billing is a natural fit for SQL: `UNIQUE` constraints give idempotency and `SUM … GROUP BY` gives metering.
Money	Integers in **paise**	Avoids floating-point errors like 0.1 + 0.2 ≠ 0.3; formatting to rupees happens only in the UI.
Frontend	HTML5, CSS3, **Bootstrap 5.3**, vanilla JavaScript (`fetch`, DOM), Chart.js (CDN)	Sem 1 stack; the console is a set of static pages served by FastAPI.
Analysis	pandas, matplotlib	Catalog requirement: monthly totals and box plot from the exported CSV.
Testing	pytest, FastAPI `TestClient`, `httpx` for the retry-storm script	Pricing is tested with the Section 15 cases before any UI.
Tools & hosting	VS Code, Git/GitHub, Render free web service (or local run + demo video)	GitHub Pages cannot run Python, so the catalog's Pages/Vercel option is replaced by Render.
Stretch	`stripe` Python library in test mode	Only after the MVP checklist passes.

Persistence decision: SQLite on the server holds everything (tenants, plans, events, invoices), because billing data must be shared, durable and checked by the server, which localStorage cannot do. The browser keeps only the signed-in key in `sessionStorage`. MongoDB Atlas is not needed: the data is naturally tables with relations and constraints.

```
meterledger/
  app/
    main.py         # FastAPI app, routers, static files
    models.py       # Pydantic request/response models
    db.py           # sqlite3 helpers, schema.sql, seed
    pricing.py      # price_tiers(), build_invoice()  (pure functions, no I/O)
    ingest.py       # validation + idempotent insert
    invoices.py     # generate / finalize / pay / void
    auth.py         # API-key and admin-key dependencies
  static/  index.html overview.html tenants.html tenant.html plans.html invoices.html
           invoice.html simulator.html portal.html  js/*.js  css/styles.css
  data/    cloud_api_usage_sample.csv  sample-data.json
  scripts/ seed.py  retry_storm.py
  analysis/report.py  charts/
  tests/   test_pricing.py  test_ingest.py  test_invoices.py
  requirements.txt  README.md  .env.example
```

8. Data model

Table	Columns	Rules
meters	id TEXT PK (api_calls, emails_sent), name, unit (call, email), aggregation ("sum")	Seeded; read-only in MVP.
plans	id TEXT PK, name, base_fee_paise INTEGER, currency ("INR"), active INTEGER, created_at	Base fee ≥ 0. A plan used on a finalized invoice cannot be edited (create a new version instead).
price_tiers	id INTEGER PK, plan_id FK, meter_id FK, from_unit INTEGER, to_unit INTEGER NULL, price_per_1000_paise INTEGER	Per plan and meter: first tier starts at 1; tiers are contiguous with no gaps or overlaps; only the last tier has to_unit = NULL (no upper limit).
tenants	id TEXT PK, name, plan_id FK, api_key_hash TEXT UNIQUE, key_prefix (first 8 chars, for display), status (active/suspended), created_at	The raw key is shown once at creation and never stored; only its SHA-256 hash is kept.
usage_events	id INTEGER PK, tenant_id FK, event_id TEXT, meter_id FK, quantity INTEGER, occurred_at TEXT (UTC ISO), period TEXT ("2026-09"), received_at TEXT	UNIQUE(tenant_id, event_id) is the idempotency guarantee. Index on (tenant_id, period, meter_id).
invoices	id INTEGER PK, number TEXT NULL, tenant_id FK, period, plan_id, status (draft/finalized/paid/void), base_fee_paise, total_paise, created_at, finalized_at, paid_at	UNIQUE(tenant_id, period) for non-void invoices; number assigned only on finalize.
invoice_lines	id INTEGER PK, invoice_id FK, meter_id, tier_label ("10,001 to 100,000"), units INTEGER, price_per_1000_paise, amount_paise INTEGER	Lines are a snapshot: later plan edits never change them.

sample-data.json (excerpt)

```
{
  "plans": [
    { "id": "free", "name": "Free", "base_fee_paise": 0,
      "tiers": { "api_calls":   [ { "from": 1, "to": 10000, "per_1000": 0 },
                                  { "from": 10001, "to": null, "per_1000": 2500 } ],
                 "emails_sent": [ { "from": 1, "to": null, "per_1000": 10000 } ] } },
    { "id": "starter", "name": "Starter", "base_fee_paise": 49900,
      "tiers": { "api_calls":   [ { "from": 1, "to": 10000, "per_1000": 0 },
                                  { "from": 10001, "to": 100000, "per_1000": 2000 },
                                  { "from": 100001, "to": null, "per_1000": 1500 } ],
                 "emails_sent": [ { "from": 1, "to": null, "per_1000": 10000 } ] } }
  ],
  "tenants": [ { "id": "t_shopkart", "name": "ShopKart", "plan_id": "starter" },
               { "id": "t_tinybot", "name": "TinyBot", "plan_id": "free" } ],
  "events": [ { "tenant_id": "t_shopkart", "event_id": "shk-2026-09-01-0001",
                "meter": "api_calls", "quantity": 500, "timestamp": "2026-09-01T04:51:10Z" } ]
}
```

Prices are per 1,000 units in paise: 2000 = ₹20 per 1,000 calls; 10000 = ₹100 per 1,000 emails (₹0.10 each).

9. API/service contract

Base URL `/api`. JSON bodies. Errors always use `{"error": "code", "detail": "human message"}`. Auth column: **T** = tenant key (`X-Api-Key`), **A** = admin key (`X-Admin-Key`), **–** = open. Admin may call tenant routes with `?tenant_id=`.

Method & path	Input	Output	Auth	Errors
`GET /me`	–	`{role, tenant?}`	T/A	401
`POST /events`	`{event_id, meter, quantity, timestamp}`	201 `{status:"accepted"}` or 200 `{status:"duplicate"}`	T	400 invalid, 409 id reused / period closed
`POST /events/batch`	`{events:[…≤1000]}`	`{accepted, duplicates, rejected:[{index, error}]}`	T	400 > 1,000 items
`GET /usage?period=2026-09`	query	`{tenant_id, period, meters:{api_calls:135400, …}}`	T/A	400 bad period
`GET /usage/daily?period&meter`	query	`[{day, quantity}]`	T/A	400
`GET /usage/estimate?period`	query	invoice preview (not saved)	T/A	–
`GET /plans`, `GET /plans/{id}`	–	plans with tiers	–	404
`POST /plans`	plan + tiers	201 plan	A	400 tier gaps/overlaps, 409 id exists
`GET /tenants`, `POST /tenants`	`{id, name, plan_id}`	list / 201 with `api_key` (shown once)	A	409 exists, 404 plan
`PATCH /tenants/{id}`	`{plan_id?, status?}`	tenant	A	404
`POST /invoices/generate`	`{period, tenant_id?}`	`{created, updated, skipped_finalized}`	A	400 future period
`GET /invoices?period&status`	query	invoice list	T/A	–
`GET /invoices/{id}`	–	invoice with lines	T/A	404 (also for another tenant's id)
`POST /invoices/{id}/finalize`	–	invoice with number	A	409 not draft
`POST /invoices/{id}/pay`	`{method:"mock"}`	invoice (paid)	A	409 not finalized
`POST /invoices/{id}/void`	`{reason}`	invoice (void)	A	409 paid
`GET /export/usage.csv?period`	query	CSV	A	–

Example: sending an event twice

```
POST /api/events                 X-Api-Key: ml_live_shopkart_****
{ "event_id": "shk-2026-09-14-0042", "meter": "api_calls", "quantity": 500,
  "timestamp": "2026-09-14T10:21:05Z" }
201 Created   { "status": "accepted", "event_id": "shk-2026-09-14-0042", "period": "2026-09" }

(same request again after a network timeout)
200 OK        { "status": "duplicate", "event_id": "shk-2026-09-14-0042" }

(same event_id, quantity 700)
409 Conflict  { "error": "event_id_reused", "detail": "event_id was already used with different data" }
```

10. Business rules

1. **Idempotency:** an event is identified by `(tenant_id, event_id)`. Insert with `INSERT … ON CONFLICT DO NOTHING`; if nothing was inserted, compare with the stored row: identical → duplicate (200), different → 409. The database constraint, not Python code, is the final guard, so concurrent retries cannot slip through.
2. **Tenant from key:** the tenant is always taken from the API key, never from the request body.
3. **Billing period:** calendar month of `occurred_at` in UTC (an event at 2026-09-30T23:59:59Z belongs to 2026-09). Events more than 5 minutes in the future are rejected.
4. **Closed periods:** once a tenant's invoice for a period is finalized, new events for that tenant and period are rejected with 409 `period_closed`.
5. **Aggregation:** monthly usage = `SUM(quantity)` grouped by tenant, meter and period. Quantity is a whole number from 1 to 1,000,000.
6. **Graduated tiers:** units fill tier 1 first, then tier 2, and so on. Each tier is charged at its own rate.
7. **Exact money:** all amounts are integers in paise. Each tier line = `units × price_per_1000_paise`, divided by 1,000 and rounded half up to a whole paisa: `(units × price + 500) // 1000`. Invoice total = base fee + sum of line amounts. No tax.
8. **Plan at generation:** a draft uses the tenant's plan at the moment it is generated. Mid-month plan changes are not prorated (out of scope).
9. **Lifecycle:** draft → finalized → paid; finalized → void. Drafts can be regenerated; finalized, paid and void invoices are never changed. A voided period can be regenerated as a new draft.
10. **Numbering:** invoice numbers `INV-202609-0001` are assigned in finalize order and never reused.
11. **Tier validation:** tiers for a meter start at 1, are contiguous, and only the last is open-ended; prices ≥ 0.
12. **Zero usage:** a tenant with no events still gets an invoice with the base fee and zero-unit lines omitted (₹0.00 invoice for Free plan tenants is allowed).

Reference code (students re-type and explain it, then extend)

```
def price_tiers(units: int, tiers: list[dict]) -> list[dict]:
    """tiers sorted by 'from'; 'to' is None for the last tier. Returns invoice lines in paise."""
    lines, remaining = [], units
    for t in tiers:
        if remaining <= 0:
            break
        size = remaining if t["to"] is None else t["to"] - t["from"] + 1
        in_tier = min(remaining, size)
        amount = (in_tier * t["per_1000"] + 500) // 1000      # round half up to 1 paisa
        lines.append({"tier": f'{t["from"]:,} to {t["to"]:,}' if t["to"] else f'{t["from"]:,}+',
                      "units": in_tier, "per_1000": t["per_1000"], "amount": amount})
        remaining -= in_tier
    return lines

def build_invoice(plan: dict, usage: dict) -> dict:
    lines = []
    for meter, units in usage.items():
        lines += [dict(l, meter=meter) for l in price_tiers(units, plan["tiers"][meter])]
    total = plan["base_fee_paise"] + sum(l["amount"] for l in lines)
    return {"base_fee": plan["base_fee_paise"], "lines": lines, "total": total}

# Idempotent insert (sqlite3)
cur = conn.execute("""INSERT INTO usage_events (tenant_id, event_id, meter_id, quantity, occurred_at,
period, received_at)
                      VALUES (?, ?, ?, ?, ?, ?, ?) ON CONFLICT(tenant_id, event_id) DO NOTHING""", row)
status = "accepted" if cur.rowcount == 1 else "duplicate"   # then compare data for the 409 case
```

11. Auth, roles and data safety

- **Keys:** each tenant gets a random key (`secrets.token_urlsafe(24)`, prefixed `ml_`) shown once at creation. The server stores only `sha256(key)` and looks tenants up by that hash. The admin key comes from the `ADMIN_KEY` environment variable in a git-ignored `.env`; `.env.example` is committed with a placeholder.
- **Role blockers:** FastAPI dependencies `current_tenant` and `require_admin` run before every protected route. Missing key → 401; tenant key on an admin route → 403; suspended tenant → 403 `tenant_suspended`.
- **Tenant isolation:** every tenant query includes `WHERE tenant_id = ?` from the key. Asking for another tenant's invoice returns 404 (not 403), so ids cannot be probed.
- **Validation:** Pydantic models enforce types, ranges and formats (event_id 1 to 64 characters of letters, digits, `-`, _; period `YYYY-MM`).
- **SQL safety:** parameterised queries only. Invoice generation runs in one transaction so a failure leaves no half-written invoice.
- **Output safety:** tenant names and ids are inserted in the page with `textContent`.
- **Browser storage:** the signed-in key is kept in `sessionStorage` and removed on Sign out; nothing sensitive in localStorage. The tenant portal shows the key masked (`ml_shop…a91f`).
- **Storage clearing:** `python scripts/seed.py --reset` rebuilds the database from `sample-data.json`. Finalized invoices are never deleted through the API; void is the only way to cancel one.
- **Stretch Stripe:** only `sk_test_…` keys, kept in `.env`; the code refuses to start with a live key.

12. Error handling and edge cases

Scenario	Expected behaviour
Same event sent twice (retry)	200 `duplicate`; usage unchanged.
Same `event_id`, different quantity or meter	409 `event_id_reused`; original kept.
Two identical requests at the same moment	Exactly one insert wins via the UNIQUE constraint; the other reports duplicate.
Unknown meter, quantity 0, negative or decimal	400 with field name, e.g. `quantity must be an integer from 1 to 1,000,000`.
Timestamp more than 5 minutes in the future / not ISO format	400 `bad_timestamp`.
Event for a finalized period	409 `period_closed` (adjustment lines are stretch).
Batch over 1,000 items	400 `batch_too_large`; nothing stored.
Batch with some bad items	Good items stored; response lists bad indexes with reasons.
Usage exactly on a tier boundary (10,000 or 100,000)	Units stay in the lower tier; next tier gets 0 and no line is printed.
Line amount with a fraction of a paisa	Rounded half up per line (Section 15, cases C4 and C5).
Tenant with no usage	Invoice with base fee only.
Generate invoices twice	Drafts updated, finalized ones reported as `skipped_finalized`; never two invoices for one tenant-period.
Finalize a paid/void invoice; pay a draft; void a paid invoice	409 with the current status in `detail`.
Plan tiers with a gap or overlap	400 `invalid_tiers` naming the bad row.
Wrong or missing key	401; UI returns to Sign in with "Key not recognised".
Server unreachable from the UI	Toast "Can't reach the billing server" with Retry.

13. Learning goals

- **Course covered:** Introduction to Computer Programming; Mathematics for AI/ML - 1; Frontend Development Basics.
- **Core learning goals:** Core (from Sem 1): Introduction to Computer Programming; Mathematics for AI/ML - 1; Frontend Development Basics. Practise HTML/CSS/JS and Python OOP and files. Domain:

Usage Metering & Billing Infrastructure. Stretch stays optional self-study, not a second degree. Extended scope: applied exploratory data analysis, metric telemetry aggregation with pandas, and visualisation with matplotlib.

By the end each student can:

1. Explain metering, idempotency and graduated pricing, and why retries cause double counting.
2. Write a pure pricing function with integer money and prove it with hand-computed test cases.
3. Design SQL tables with primary keys, foreign keys and UNIQUE constraints, and aggregate with `SUM … GROUP BY`.
4. Build a REST API with FastAPI: routes, Pydantic validation, status codes, header-key auth and tenant isolation.
5. Model a state machine (draft → finalized → paid / void) and reject invalid transitions.
6. Build a multi-page Bootstrap console that calls the API with `fetch` and draws charts with Chart.js.
7. Use pandas to group and summarise usage, and read a box plot (median, quartiles, outliers).
8. Write pytest tests and a load-style script, and deploy a Python web app.

New technologies to learn this month: FastAPI + Uvicorn, Pydantic, SQLite/SQL, pytest, httpx, Chart.js, Render deployment (Stripe test mode only as stretch). Everything else is from Sem 1.

14. Milestones

Week	Learn	Build	Checkpoint
Week 1: Pricing & data	Usage-based pricing and tiers; why money uses integers; Python functions, dicts and tests (pytest); SQL basics (CREATE, INSERT, SELECT, GROUP BY, UNIQUE); Git branching	One-page brief; repo and README skeleton; `pricing.py` with all six Section 15 cases as tests; `schema.sql`; `seed.py` that loads plans, tenants and events from the CSV; SQL query that prints monthly totals	Six pricing tests green; seed totals printed per tenant
Week 2: Ingestion API	HTTP, REST, JSON and status codes; FastAPI routes, Pydantic, dependencies; environment variables; hashing a key with SHA-256	Auth dependencies; `POST /events` and `/events/batch` with idempotency; usage, daily and estimate endpoints; tenants and plans endpoints; `retry_storm.py`	Retry storm passes with zero double counting; all endpoints work from `/docs`
Week 3: Invoices & console	State machines; SQL transactions; `fetch` with headers; Bootstrap tables, modals, badges; Chart.js bars	Invoice generate / finalize / pay / void; S1 to S7 screens; tenant portal S9; all UI states	Journeys: send events → generate → finalize → pay work in the browser
Week 4: Analysis & ship	pandas (`read_csv`, `groupby`, `pivot_table`), box plots in matplotlib; deployment on Render; README writing	CSV export; `analysis/report.py` (3 charts); S8 simulator; responsive and accessibility pass; deploy; demo video	Checklist in Section 15 passes; live URL or video

Rule of thumb each week: mornings for the "Learn" column, afternoons for "Build". Stretch (Stripe test mode, volume pricing) starts only after the Week 3 checkpoint is fully green.

15. Evaluation

Catalog metrics and how they are measured

Catalog metric	Measured how	Pass when
Zero double-counted usage events across 100k simulated retries	`scripts/retry_storm.py --seed 7`: makes 20,000 unique events for 4 tenants, sends each 1 to 9 times (about 100,000 sends) in shuffled order, through batch calls from 4 threads at once. Then compares `/usage` totals with the sum of the unique events.	Totals equal exactly; `usage_events` has exactly 20,000 rows; report prints accepted = 20,000
Invoice calculation matches hand-computed cases 100%	pytest runs cases C1 to C6 below through `build_invoice` and through the API.	6 of 6 match to the paisa
Expected output: 2 to 3 telemetry charts	`python analysis/report.py data/usage.csv`	Saves 3 PNGs in `charts/`

Hand-checked worked example

Plans used. *Starter*: base fee ₹499; API calls 1 to 10,000 free, 10,001 to 100,000 at ₹20 per 1,000, above 100,000 at ₹15 per 1,000; emails ₹100 per 1,000 (₹0.10 each). *Free*: base fee ₹0; API calls 1 to 10,000 free, above that ₹25 per 1,000; emails ₹100 per 1,000.

Case C1, ShopKart on Starter, September 2026: 135,400 API calls and 2,350 emails.

Line	Units	Rate	Working (in paise)	Amount
Base fee	–	–	49,900	₹499.00
API calls 1 to 10,000	10,000	₹0 / 1,000	10,000 × 0 ÷ 1,000 = 0	₹0.00
API calls 10,001 to 100,000	90,000	₹20 / 1,000	90,000 × 2,000 ÷ 1,000 = 180,000	₹1,800.00
API calls 100,001+	35,400	₹15 / 1,000	35,400 × 1,500 ÷ 1,000 = 53,100	₹531.00
Emails	2,350	₹100 / 1,000	2,350 × 10,000 ÷ 1,000 = 23,500	₹235.00
Total			49,900 + 0 + 180,000 + 53,100 + 23,500 = 306,500	₹3,065.00

Check that the units add up: 10,000 + 90,000 + 35,400 = 135,400 ✔.

Case	Situation	Working	Expected total
C1	ShopKart, Starter, 135,400 calls, 2,350 emails	Table above	₹3,065.00
C2	TinyBot, Free, 8,200 calls, 0 emails	All 8,200 in the free tier; base ₹0	₹0.00
C3	Starter, exactly 10,000 calls, 0 emails (boundary)	10,000 in tier 1 (free); tier 2 gets 0 units, no line	₹499.00
C4	Starter, 100,001 calls (boundary + rounding)	Tier 2: 90,000 × 2,000 ÷ 1,000 = 180,000 p. Tier 3: 1 × 1,500 = 1,500 → (1,500 + 500) // 1,000 = 2 p. Total 49,900 + 180,000 + 2 = 229,902 p	₹2,299.02
C5	Free, 12,345 calls (rounding)	Tier 2: 2,345 × 2,500 = 5,862,500 →	₹58.63

Case	Situation	Working	Expected total
		(5,862,500 + 500) // 1,000 = 5,863 p	
C6	ShopKart sends event `e-1001` (500 calls) three times, plus `e-1002` (300 calls) once, on Starter with no other usage	Counted once: 500 + 300 = 800 calls, all in the free tier	₹499.00 (usage 800, not 1,800)

Acceptance criteria / demo checklist

1. Cases C1 to C6 pass in pytest and the same totals appear on the invoice detail screen after seeding those events.
2. The retry storm finishes with zero double counting and the simulator screen shows the counts.
3. Re-sending an event returns `duplicate`; changing its quantity returns 409.
4. Generating invoices twice still shows one invoice per tenant for the month.
5. A finalized invoice gets a number, cannot be regenerated, and blocks new events for that period (409).
6. Draft → finalized → paid works; paying a draft or voiding a paid invoice is refused with a clear message.
7. A tenant key sees only its own usage and invoices; another tenant's invoice id returns 404; admin routes return 403.
8. `report.py` produces: (1) monthly API calls per tenant (bar), (2) box plot of monthly usage per tenant across months (catalog requirement), (3) daily total usage over time (line).
9. All screens work at 375 px phone width and on a laptop; money always shows two decimals.

Area	Points	Full marks when
Pricing & invoice correctness	25	C1 to C6 exact; rounding and boundaries explained in README
Ingestion & idempotency	20	Retry storm passes; validation codes as in Section 12
API, database & roles	15	Schema and endpoints match Sections 8 to 9; tenant isolation holds
Frontend console & portal	15	Screens S1 to S9 with all UI states, responsive
Data analysis	10	3 correct, labelled charts including the box plot
Code quality, tests & Git	10	pytest suite; both students commit; no secrets in repo
Docs & demo	5	README, deployed link or video

16. Risks and assumptions

Risk	Likelihood	Impact	Mitigation
Floating-point money errors	High	High	Paise integers everywhere; rounding rule fixed in Section 10; C4/C5 tests
Students try to add Kafka, real Stripe or taxes	Medium	High	Out-of-scope list; Stripe test mode only after Week 3 checkpoint
Idempotency done in Python only, races slip through	Medium	High	UNIQUE constraint required; retry storm uses 4 threads
SQLite slow for 100,000 sends	Low	Medium	Batch endpoint (500 per call) and one transaction per batch
FastAPI and SQL are new	Medium	Medium	Week 1 SQL on seed data, Week 2 FastAPI

Risk	Likelihood	Impact	Mitigation
			tutorial; `/docs` for testing
Tier logic wrong at boundaries	Medium	High	C3 and C4 boundary tests written before the API
Free Render instance sleeps or resets SQLite	Medium	Low	Seed on startup; demo video as backup

Assumptions: one currency (INR) and no GST; billing periods are UTC calendar months; the AWS/GCP metering CSV is simulated, so tenant names and numbers are illustrative; each tenant is on exactly one plan per invoice.

17. Deliverables

- GitHub repo with clean commits from both students and a passing pytest suite
- Deployed demo on Render (or local run + recorded walkthrough)
- README: setup (`pip install -r requirements.txt`, `python scripts/seed.py`, `uvicorn app.main:app`), how metering, idempotency and tiers work with case C1, data model, API table, roles, screenshots
- `sample-data.json`, the source CSV and an exported `usage.csv`
- `scripts/retry_storm.py` output saved as `retry_storm_result.txt`
- `analysis/report.py` and the 3 chart PNGs
- Demo video (3 to 5 minutes) or live demo

18. References

- Reference docs: MDN Web Docs (HTML, CSS, JS, `fetch`, HTTP status codes); Python 3 tutorial (functions, dicts, classes, files) and docs for `sqlite3`, `hashlib`, `secrets`; Bootstrap 5 docs
- FastAPI official tutorial (First steps, Request body, Dependencies, Handling errors, Testing); pytest "Get started"; httpx quickstart
- SQLite docs: UNIQUE constraints, UPSERT (`ON CONFLICT`), aggregate functions; Chart.js bar charts; pandas "10 minutes to pandas"; matplotlib `boxplot`
- Stripe docs: "Usage-based billing" overview, graduated vs volume tiers, idempotent requests (concept reading; test mode for stretch)
- Render docs: deploy a FastAPI app
- Dataset: AWS/GCP Simulated API Usage Metering CSV
- Research paper / foundational spec: Cloud Metering and Billing Architectures (NIST Cloud Computing Special Publication 500-292)

19. Sign-off

Role	Name	Date	Approved
Student 1 (ingestion & database)			
Student 2 (invoices UI & analysis)			
Mentor			
Program reviewer			

What was scoped down from v0.1 and why

- The catalog's front-end-only stack (localStorage, GitHub Pages) became a **real backend** (FastAPI + SQLite) because this is an Application Development project and billing data must be checked by a server; hosting moved to Render since GitHub Pages cannot run Python.
- "No Kafka/Kinesis" is kept: events arrive over plain HTTP, with a batch endpoint instead of a stream.
- "Free-tier then paid slice" moved from Good to have into the MVP as **graduated tiers**, because it is the same function as "units × price" (a single tier) and is what the hand-checked cases test.
- The 100k-retry metric is met with 20,000 unique events sent about 100,000 times through the batch endpoint, so it runs in minutes on a laptop.
- Real payments, taxes, multiple currencies and proration are out of scope; payment is a mock button and Stripe is stretch in test mode only.

- The data-analysis item appeared in Good to have, Stretch and Evaluation. It is now a **small required script** (3 charts including the box plot) in Week 4, with deeper EDA as stretch.
- Month 1 to Month 3 milestone rows replaced by a Week 1 to Week 4 learn-and-build plan.